

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Siriyala Mubina	Reg. No.:	192324139
Section / Batch:	B.Tech/AIDS 2023	Date:	23/08/26

Code of Conduct

I Siriyala Mubina(192324139)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Mubina

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Design an algorithm to perform **Reference Resolution** by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

2. Develop an algorithm to analyze **Text Coherence** and **Discourse Structure** by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

3. Design an algorithm for a **Conversational Agent** that identifies **Dialogue Acts** from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?"

Agent: "Sure, where would you like to travel?"

User: "I want to go to Chennai."

Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

4. Design an algorithm for **Language Generation** that converts a semantic representation into a grammatically correct sentence through **Surface Realization**. Consider the semantic input:

Action: Buy
Agent: Student
Object: Book
Tense: Past

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

5. Design an algorithm that combines **Interlingua-based Machine Translation** and **Statistical Translation Approaches**. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. Analyze the source sentence.
2. Create an Interlingua representation.

3. Generate candidate target-language translations.
4. Apply statistical scoring to select the most appropriate translation.
5. Produce the final translated sentence.

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question

1. Design an algorithm to perform **Reference Resolution** by identifying pronouns and linking them to the correct entities in a discourse. Apply your algorithm to the text:

"Ravi met Arun at the library. He borrowed a book and later returned it."

Generate the resolved discourse by replacing pronouns with their corresponding entities. Further, validate the correctness of the resolution process by explaining how discourse context is used to identify the antecedents.

Algorithm Steps:

1. Tokenize the discourse into sentences and extract entities (proper nouns) and pronouns using POS tagging.
2. Maintain a discourse entity list ordered by recency, each tagged with gender, number and animacy features.
3. For every pronoun found, filter candidate antecedents from the entity list by gender/number/animacy agreement.
4. Apply syntactic-role and recency heuristics (prefer the most recently mentioned entity that was in subject position) to rank the remaining candidates.
5. Select the highest-ranked candidate as the antecedent and link/replace the pronoun with it.
6. Output the resolved discourse together with the antecedent mapping.

Pseudocode:

```
FUNCTION ResolveReferences(discourse):
    sentences <- SPLIT(discourse, '.')
    entityList <- []           // discourse history, most recent first
    resolvedText <- ""
    FOR each sentence in sentences:
        tokens <- TOKENIZE(sentence)
        FOR each token in tokens:
            IF isProperNoun(token):
                entityList.INSERT_FRONT(token)    // new entity introduced
            ELSE IF isPronoun(token):
                candidates <- [e IN entityList WHERE agrees(e, token)]
                antecedent <- candidates[0]        // nearest agreeing entity
                token <- antecedent                // replace pronoun
            resolvedText <- resolvedText + REBUILD(tokens)
    RETURN resolvedText
```

Application to the Given Example:

Text: "Ravi met Arun at the library. He borrowed a book and later returned it."

Entities introduced in sentence 1: Ravi (subject/agent), Arun (object), "the library" (location).

“He”: agrees in gender and number with both Ravi and Arun; the subject-continuity heuristic favours the most recent subject/agent, so He → Ravi.

“it”: the only inanimate entity introduced before “it” appears is “a book”, so it → book.

Resolved discourse: “Ravi met Arun at the library. Ravi borrowed a book and later returned the book.”

Evaluation / Validation:

Discourse context supplies three cues used by the algorithm: (a) grammatical agreement — “he” is masculine singular, matching both Ravi and Arun, and “it” is inanimate singular, matching “a book”; (b) recency — the algorithm keeps a stack of recently mentioned entities and prefers the nearer one; and (c) grammatical-role continuity (centering) — an entity that was the subject of one clause tends to remain the topic/subject of the next. Since Ravi was the subject of sentence 1 and no competing male subject intervenes, “He” correctly resolves to Ravi, and since “a book” is the sole inanimate candidate, “it” correctly resolves to book.

Question

2. Develop an algorithm to analyze **Text Coherence** and **Discourse Structure** by identifying logical relationships between sentences such as cause-effect, sequence, and elaboration. Apply your algorithm to the following discourse:

"The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

Identify the discourse relations among the sentences and construct a discourse structure representation. Further, justify how these relations contribute to maintaining coherence in the text.

Algorithm Steps:

1. Segment the discourse into elementary discourse units (EDUs), typically one per sentence.
2. Scan each EDU for discourse connectives / cue phrases (e.g. "therefore", "because", "then", "however").
3. Classify the relation between each pair of consecutive EDUs using the cue phrase; if no cue phrase is present, fall back to semantic/temporal similarity heuristics.
4. Link the EDUs by their relation labels to build a discourse tree/graph.
5. Judge overall coherence: the text is coherent if every EDU is connected to the structure by a valid relation (no isolated, unrelated EDU).

Pseudocode:

```
FUNCTION AnalyzeCoherence(discourse):  
  EDUs <- SPLIT_INTO_SENTENCES(discourse)  
  relations <- []  
  FOR i = 1 TO LENGTH(EDUs) - 1:  
    cue <- FIND_CUE_PHRASE(EDUs[i+1])
```

```

IF cue IN {"therefore","so","thus":
  relation <- "Cause-Effect"
ELSE IF cue IN {"then","after","later"}:
  relation <- "Sequence"
ELSE IF cue IN {"however","but"}:
  relation <- "Contrast"
ELSE:
  relation <- "Elaboration"      // default when topics overlap
relations.APPEND( (EDUs[i], EDUs[i+1], relation) )
RETURN BUILD_DISCOURSE_TREE(relations)

```

Application to the Given Example:

Text: "The roads were flooded after heavy rainfall. Therefore, schools were closed for the day. Students attended classes online."

EDU1: "The roads were flooded after heavy rainfall."

EDU2: "Therefore, schools were closed for the day." — cue word "Therefore" →
 $\text{relation}(\text{EDU1} \rightarrow \text{EDU2}) = \text{Cause-Effect}$.

EDU3: "Students attended classes online." — no explicit cue, but it is a direct consequence of EDU2 →
 $\text{relation}(\text{EDU2} \rightarrow \text{EDU3}) = \text{Elaboration/Result}$.

Discourse structure: EDU1 --Cause--> EDU2 --Result/Elaboration--> EDU3 (a single connected chain rooted at the flooding event).

Evaluation / Validation:

Each EDU links to its neighbour through an explicit or inferable relation, so the passage forms one connected discourse structure rather than a set of unrelated statements. The cue word "Therefore" makes the cause-effect link explicit, and EDU3 coheres because it logically follows from "schools closed" (an entailed elaboration — an alternative arrangement for the closed schools). This single unbroken chain of relations, all centred on one topic (flooding → closure → online classes), is what makes the passage coherent.

Question

3. Design an algorithm for a **Conversational Agent** that identifies **Dialogue Acts** from user interactions. Apply your algorithm to the conversation:

User: "Can you book a train ticket for me?"

Agent: "Sure, where would you like to travel?"

User: "I want to go to Chennai."

Agent: "Your ticket has been booked."

Classify each utterance as Request, Question, Inform, Confirmation, or Action. Generate the dialogue-act sequence and evaluate how the identified acts help the conversational agent interpret user intentions.

Algorithm Steps:

1. Tokenize each utterance.
2. If the utterance contains a request marker such as "can you" and ends with "?", classify it as Request.
3. Else if the utterance ends with "?", classify it as Question.
4. Else if the utterance contains an information marker such as "I want", "I would like", "I have", classify it as Inform.
5. Else if the utterance contains an acknowledgement/completion marker such as "sure", "has been", "done", "booked", classify it as Confirmation.
6. Otherwise default to Inform; append the label to the dialogue-act sequence.

Pseudocode:

```
FUNCTION ClassifyDialogueAct(utterance):
  IF utterance CONTAINS "can you" AND utterance ENDSWITH "?":
    RETURN "Request"
  ELSE IF utterance ENDSWITH "?":
    RETURN "Question"
  ELSE IF utterance CONTAINS ("I want" OR "I would like" OR "I have"):
    RETURN "Inform"
  ELSE IF utterance CONTAINS ("sure" OR "has been" OR "done" OR "booked"):
    RETURN "Confirmation"
  ELSE:
    RETURN "Inform"           // default

FUNCTION AnalyzeConversation(turns):
  sequence <- []
  FOR each turn in turns:
    sequence.APPEND( ClassifyDialogueAct(turn) )
  RETURN sequence
```

Application to the Given Example:

User: "Can you book a train ticket for me?" → contains "can you" + "?" → Request

Agent: "Sure, where would you like to travel?" → "Sure" is a Confirmation marker, followed by a question seeking the destination slot → Confirmation + Question

User: "I want to go to Chennai." → contains "I want" → Inform

Agent: "Your ticket has been booked." → contains "has been" → Confirmation / Action

Dialogue-act sequence: Request → Confirmation+Question → Inform → Confirmation/Action.

Evaluation / Validation:

The dialogue-act sequence lets the agent track conversation state: the Request opens a booking task, the Question fills the missing destination slot, the Inform supplies that slot value, and the closing Confirmation/Action signals task completion. Recognising these acts lets the system decide what to do next — ask another question, invoke the booking service, or end the dialogue — instead of just reacting to raw text.

Question

4. Design an algorithm for **Language Generation** that converts a semantic representation into a grammatically correct sentence through **Surface Realization**. Consider the semantic input:

Action: Buy
Agent: Student
Object: Book
Tense: Past

Generate the final natural language sentence and explain how the algorithm performs lexical selection, sentence structuring, and surface realization. Further, validate the grammatical correctness of the generated output.

Algorithm Steps:

1. Content determination: take the semantic frame (Action, Agent, Object, Tense) as the content to express.
2. Sentence planning: choose sentence structure Subject(Agent) - Verb(Action) - Object(Object) and map Tense to the required verb form.
3. Lexicalization: choose surface words for each semantic slot (Agent -> "student", Action -> "buy", Object -> "book").
4. Morphological / surface realization: apply tense inflection (buy -> bought for past tense), insert the correct article ("a book"), and enforce subject-verb agreement.
5. Linearize the chosen tokens into a single grammatical sentence.

Pseudocode:

```
FUNCTION SurfaceRealize(frame):  
  subject <- LEXICALIZE(frame.Agent)           // "student" -> "The student"  
  verb   <- INFLECT(frame.Action, frame.Tense) // Buy + Past -> "bought"  
  object <- LEXICALIZE_WITH_ARTICLE(frame.Object) // "book" -> "a book"  
  sentence <- subject + " " + verb + " " + object + "."  
  RETURN sentence
```

Application to the Given Example:

Input: Action: Buy, Agent: Student, Object: Book, Tense: Past

subject = "The student", verb = INFLECT("Buy", "Past") = "bought", object = "a book"

Generated sentence: "The student bought a book."

Evaluation / Validation:

Lexical selection mapped every semantic role to an appropriate open-class word (student, buy, book); sentence structuring placed them in canonical English Subject-Verb-Object order; surface realization then applied past-tense inflection and the indefinite article. The result is grammatically correct — subject-verb agreement holds and the sentence expresses exactly the semantic content of the input frame, with no missing or extra roles.

Question

5. Design an algorithm that combines **Interlingua-based Machine Translation** and **Statistical Translation Approaches**. Apply your algorithm to translate the sentence:

"The boy is playing football."

Perform the following steps:

1. Analyze the source sentence.
2. Create an Interlingua representation.
3. Generate candidate target-language translations.
4. Apply statistical scoring to select the most appropriate translation.
5. Produce the final translated sentence.

Finally, evaluate how the Interlingua representation and statistical methods improve translation quality and reduce ambiguity.

Algorithm (applied step by step as requested):

1. Analyze the source sentence: POS-tag and parse “The boy is playing football” → Subject: boy, Verb: play (present continuous), Object: football.

2. Create an Interlingua representation: a language-neutral semantic frame independent of source/target syntax: PLAY(Agent: BOY, Object: FOOTBALL, Tense: PRESENT_CONTINUOUS).

3. Generate candidate target-language translations from the interlingua frame using target-language generation rules. For French: C1: “Le garçon joue au football.” C2: “Le garçon est en train de jouer au football.” C3: “Le garçon joue le football.” (incorrect article/preposition).

4. Apply statistical scoring (e.g. n-gram language-model probability) to rank the candidates: C1 scores highest because “jouer au football” is the statistically frequent, idiomatic collocation; C3 is penalised for the wrong preposition/article, and C2 is penalised for being a longer, less-frequent paraphrase.

5. Produce the final translated sentence: “Le garçon joue au football.”

Pseudocode:

```
FUNCTION InterlinguaStatisticalMT(sourceSentence):  
  parseTree <- PARSE(sourceSentence)  
  interlingua <- MAP_TO_INTERLINGUA(parseTree)    // language-neutral frame  
  candidates <- GENERATE_CANDIDATES(interlingua, targetLang)  
  FOR each c in candidates:  
    c.score <- LANGUAGE_MODEL_SCORE(c)           // statistical scoring  
  best <- ARGMAX(candidates, by = score)  
  RETURN best
```

Evaluation:

The interlingua stage abstracts away from source-language-specific syntax, so the same semantic frame could generate correct output in any target language without a direct source-to-target rule set — this

reduces ambiguity because the meaning (who plays what, and when) is fixed before any word choice is made. The statistical scoring stage then resolves the remaining surface-level ambiguity by preferring the candidate with the highest corpus-based probability, improving fluency and overall translation quality over a purely rule-based candidate generator.